



ISLAMIC UNIVERSITY OF TECHNOLOGY

The Organization of Islamic Cooperation(OIC)

Department of Electrical and Electronic Engineering (EEE)

Project Report

EEE 4602
Section: **C - 2**

Student ID	Student Name
210021308	Adhnan Kalim (c)
210021330	Ar- Rafi Ishraq
210021342	Ashfiq Ul Rahman
210021346	Tanjim Mahbub

Introduction:

Convolution is a mathematical operation that expresses the relationship between the input, the impulse response of a system, and the output. It is fundamental in linear time-invariant (LTI) systems, where the system's output is completely determined by its impulse response and the input signal. Signal processing, image analysis, and machine learning all make extensive use of convolution, a basic idea in mathematics and engineering. In order to create a feature map that emphasizes the similarities between the two inputs, two functions are combined mathematically by sliding one function over the other, element-by-element.

The integral of the product of two functions, one of which is shifted and reversed, is another way to define it. Convolution is a technique used in signal processing to filter signals, eliminate noise, and extract pertinent features. Convolution is used in image analysis for feature extraction, edge detection, and image filtering. Convolutional neural networks (CNNs) are used in machine learning to extract features from videos and images using convolutional layers.

For continuous time signal,

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau) d\tau$$

For discrete time signal,

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

Brief Summary of the Problem Statement:

This project consists of designing Graphical User Interface (GUI) that enables users to view and carry out the convolution operation between two signals in either the continuous-time or discrete-time domains. The GUI has to be designed in such a way that animation speed can be adjusted and a reset option needs to be made to restart the process.

The design also contains manual selection of the input and signal modes; so that the user can select the custom signals, and use real-time animation to watch the convolution process.

- In discrete mode, users supply signal vectors and the matching initial indices in discrete mode.
- In continuous mode, users define parameters like start time, end time, and amplitude and choose signal types (such as impulse, step, triangular, rectangular, or sawtooth).

Problem Review:

To design and implement a GUI application for visualizing convolution between two signals, one must have a solid understanding of several basic concepts in systems and signals as well as Programming language (MATLAB or Python) and GUI.

1.Signal Representation:

A clear understanding of how signals are represented in both discrete and continuous domains is essential.

- **Discrete-Time Signals** are only present at specific time steps, usually at integer values (e.g., $n = -2, -1, 0, 1, 2$, etc.). They are represented by the computer as lists or arrays of numbers, such as

$x[n]=[1,2,3]$. Each value in the list corresponds to a signal value at a particular time interval which is defined as 'n'. These signals, as well as a starting index that specifies when the signal begins, can be manually entered by the user into the GUI.

- **Continuous time signals** are defined at every moment in time, not just at specific steps; such as: step signals, triangular waves, and sine waves. Since a computer can't handle an infinite number of points, we approximate continuous signals by breaking them into small time steps (sampling). In the GUI, users can choose the type of signal (like step or triangular) and set values like: start time, end time & amplitude.

- **The impulse response** is a special signal that tells us how the system reacts to a short, sharp input (called a delta or impulse). Once we know this response, we can use it to find the output for any other input using convolution. The user provides this impulse response just like the input signal—either by typing values (for discrete) or choosing a shape and parameters (for continuous).

2.Convolution concepts:

The idea of convolution is essential to the analysis of linear time-invariant (LTI) systems. Both discrete-time and continuous-time convolution must be understood, including how to flip, shift, and multiply signals as well as how to carry out the integration or summation that produces the final result.

For continuous time signal,

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau) d\tau$$

For discrete time signal,

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

3.Tools and Programming Knowledge:

- Knowledge of MATLAB coding (as we are using MATLAB).
- App Designer for GUI.
- Built-in **conv()** function for convolution.
- Plotting tools (**stem, plot, etc.**) for visualization.
- Loop, time interval and pause() logic to implement animation effect.

4.Input Validation and Error Handling:

Every interactive application needs to check user input and handle errors; so that the user has a reliable and safe experience. Users must provide a variety of inputs for this convolution visualization project. These inputs include impulse responses, discrete signal vectors, and continuous-time signal parameters such as amplitude, start and end times, and sampling resolution. If inputs are not properly checked, they can cause the program to behave in unexpected ways, make plots that are wrong, or even crash. To avoid these problems, the GUI was designed to carefully check all user input before doing any calculations or showing any data.

Methodology:

1. Basic Convolutional Code:

The fundamental code of convolution was developed at first. Different functions were developed for both continuous-time and discrete-time convolution. A for-loop structure that mirrored the summation formula was used to implement convolution in discrete time. Numerical integration techniques were then used to simulate the convolution integral with sampled signals in continuous time. These basic functions were tested using predefined inputs to ensure that their outputs matched the expected results.

2. Applying GUI to the code:

A clear and simple user experience was intended for the graphical user interface. There were two primary sections to it. Users can choose between Continuous-time and Discrete-time signals, and do Convolution or Correlation as per user choice and adjust the animation speed using the Slider, which is located on the left bottom side. The operation is visually simple to follow thanks to the three synchronized subplots on the right side of the Visualization Panel that show the input signal, the shifting impulse response, and the output signal that results.

3. Adding animation:

Mainly, Loop is used with 'pause()' function to create animation effect. Time-interval was used inside the loop for

this purpose. By adjusting the animation speed manually, the steps could be shown more quickly or more slowly, which gave a clear idea of the operation.

4.Adding correlation:

In addition to convolution, correlation was used as an additional tool. Users could now alternate between convolution and correlation operations and see how the outcomes varied. With only minor adjustments, the majority of the current convolution logic could be utilized again because the mathematical procedure is similar, with the main distinction being that there is no time reversal in correlation. The tool became more complete for students studying signal processing with this addition.

5.Debugging:

Debugging was done in each step to find out the errors in the simulation. It consisted of identifying and fixing issues; such as: improper signal plotting, incorrect convolution results for certain edge cases, invalid input handling, and GUI glitches.

6.Compiling smooth working code for simulation:

After debugging, all modules were integrated into a single script. At this stage, responsive GUI requirements as well as lag-free animations were prioritized alongside general performance optimization. In addition, maintenance and future edits were taken into consideration. The final integration step was the interactive and functional oscillation simulation tool.

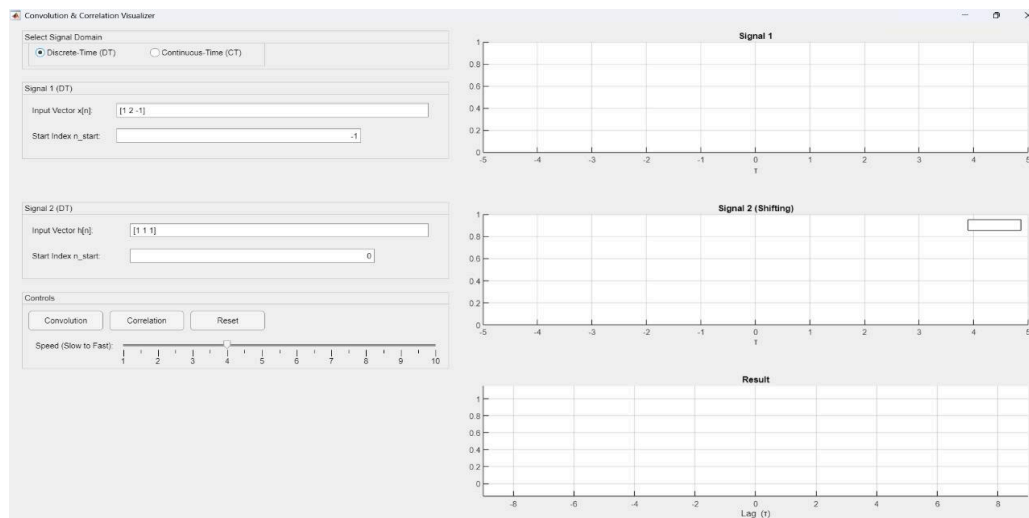
7.Special feature:

Correlation and the controlling of adjusting animation speed are the additional features.

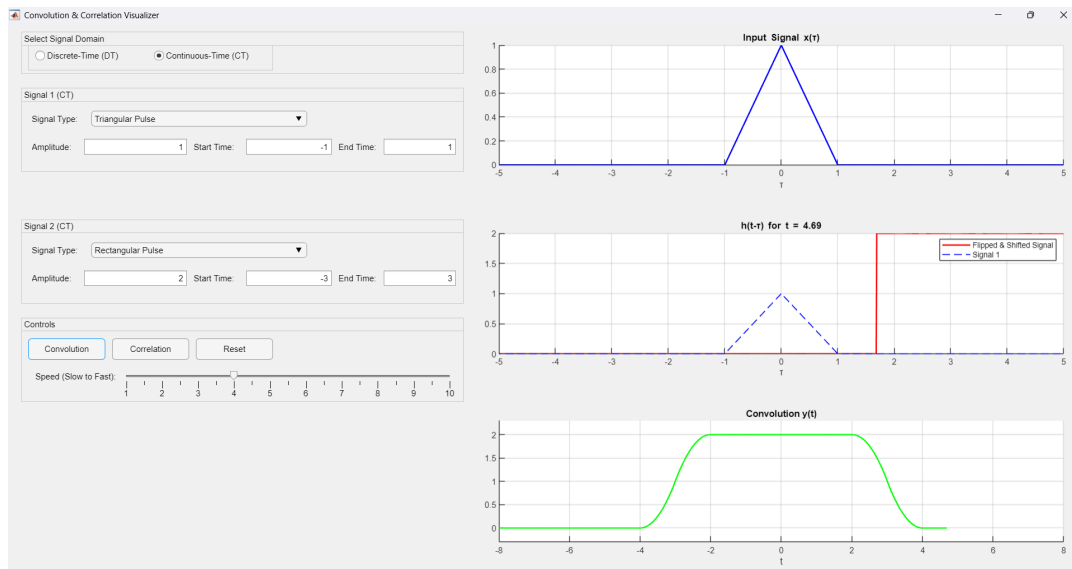
Design:

This is our GUI interface Design:

Discrete-time signal GUI:



Continuous-time signal GUI:



This Convolution and Correlation Visualizer GUI is designed to provide an intuitive and interactive learning experience for understanding fundamentals of convolution and correlation. Here, in the interface every option is clearly labeled so that sections for mode selection (discrete or continuous), operation type (convolution or correlation), and signal parameters can be easily accessed by the user. In **discrete mode**, there is a section named **Input** where a signal can be customized with specified start indices. On the other hand, in **continuous mode**, there is an option for choosing different waves like **impulse**, **step**, **rectangular**, **triangular**, and **sawtooth** with specified adjustable parameters.

The output segment shows three synchronized plots: the input signal, the transformed impulse response and the growing result of the operation. Also there is an adjustable speed slider which allows the user to control the animation speed. Thus this GUI animates the step by step process of each operation.

Tool Used:

We utilized **MATLAB App Designer** for the complete project. There are several pros of using MATLAB for this task and its suitability. **MATLAB** was chosen due to its strengths in:

- Signal processing
- Mathematical computations
- Graphical visualizations

App Designer simplified:

- Building interactive GUIs
- Integrating built-in plotting tools and control elements

Gained hands-on experience in:

- Creating animated real-time visualizations for **convolution** and **correlation**
- Managing user interface elements
- Validating user inputs
- Controlling dynamic plot updates using MATLAB's animation routines and GUI architecture

Improved **debugging skills** through:

- Tackling unexpected behaviors and logical issues
- Handling dynamic signals and multiple user inputs
- Systematic testing of the program to resolve UI and logic problems

Overall project outcomes:

- Significantly enhanced MATLAB programming skills
- Increased confidence in designing **clear and simple user interfaces** for engineering applications

Team Contribution:

Student Id	Student Name	Contributions
210021308	Adhnan Kalim	1) Adding Correlation
		2) Animation effects
		3) GUI designing
210021330	Ar- Rafi Ishraq	1) Code writing
		2) Debugging
		3) Report writing
210021342	Ashfiq UI Rahman	1) Report writing
		2) Adding Correlation
		3) Animation effects
210021346	Tanjim Mahbub	1) Code writing
		2) Debugging
		3) GUI designing

EEE 4602 TIMELINE

PROJECT ANNOUNCEMENT

PROJECT PLANNING
&
RESEARCH

WEEK
1

IMPLEMENT
DISCRETE-TIME
SIGNAL HANDLING

WEEK
2

IMPLEMENT
CONTINUOUS-TIME
SIGNAL HANDLING

WEEK
3

GUI DEVELOPEMENT
&
INTERACTION DESIGN

WEEK
4

ADD ANIMATION,
SPEED CONTROL

WEEK
5

DEBUGGING
DOCUMENTATION
FINALIZATION

WEEK
6

PROJECT SUBMISSION